

4. domača naloga: Odkrivanje robov v slikah z diskretno Fourierovo transformacijo

Diskretna Fourierova transformacija je zelo uporabna pri analizi in obdelavi slik, saj lahko z njeno pomočjo analiziramo strukture slike, identificiramo vzorce, filtriramo, in slike tudi stiskamo. V tej nalogi se bomo osredotočili na odkrivanje robov v sliki. Odkrivanje robov je eden od osnovnih postopkov v računalniškem vidu, ki služi kot predprocesiranje za nadaljnje obdelave, kot je detekcija objektov na sliki.

Diskretna Fourierova transformacija

Diskretna Fourierova transformacija (DFT) v osnovi preslika diskreten signal iz časovne v frekvenčno domeno:

$$X_n = \sum_{k=0}^{N-1} x_k \cdot e^{-i2\pi \frac{n}{N}k} \quad , \quad \text{za } n \in [0, N-1] \quad , \quad (1)$$

kjer so x_k vrednosti v časovni domeni, X_n pa so koeficienti v frekvenčni domeni, ki vsebujejo informacijo o amplitudi in fazi vsake frekvenčne komponente.

Podobno z inverzno Fourierovo transformacijo (IDFT) preidemo iz frekvenčne domene nazaj v časovno domeno:

$$x_k = \frac{1}{N} \sum_{n=0}^{N-1} X_n \cdot e^{i2\pi \frac{k}{N}n} \quad , \quad \text{za } k \in [0, N-1] \quad . \quad (2)$$

Dvodimenzionalna diskretna Fourierova transformacija in njen inverz

Pri slikah ne moremo govoriti o signalu v časovni in frekvenčni domeni, vendar lahko kljub temu uporabimo koncept preslikave z diskretno Fourierovo transformacijo – govorimo o preslikavi iz prostorske v Fourierovo domeno. V nalogi bomo obravnavali sivinske slike, ki so predstavljene kot 2D mreža slikovnih točk. Vsaka točka v sliki ima svojo vrednost (intenziteto), ki je kar nivo sivine v območju $[0, 1]$. Vrednosti v Fourierovi domeni podajajo informacijo o posameznih gradnikih, ki sestavljajo sliko. Podobno kot lahko pri zvočnem signalu v Fourierovi domeni razberemo višino tonov (frekvenco) v signalu, lahko z analizo slike v Fourierovi domeni ugotovimo, kje so gladki prehodi med posameznimi deli slike (na primer popolnoma bela slika) in kako izraziti in pogosti so ti prehodi (na primer šahovnica). Govorimo lahko o stopnji sprememb, ki pri slikah odigra podobno vlogo kot frekvence pri analizi časovnih signalov.

Ker so slike dvodimenzionalne, definiramo dvodimenzionalno diskretno Fourierovo transformacijo:

$$F_{n,m} = \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} f_{k,l} \cdot e^{-i2\pi \left(\frac{n}{N}k + \frac{m}{M}l \right)} \quad , \quad (3)$$

kjer je $f_{k,l}$ vrednost slikovne točke na koordinatah (k, l) v prostorski domeni, $F_{n,m}$ pa predstavlja koeficiente v Fourierovi domeni na koordinati (n, m) . V prostorski in Fourierovi domeni prva koordinata podaja vrstico, druga pa stolpec točke. V enačbi smo vzeli, da je N višina in M širina slike.

Izkaže se, da lahko 2D DFT spremenimo v zaporedje dveh 1D DFT:

$$F_{n,m} = \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} f_{k,l} \cdot e^{-i2\pi(\frac{n}{N}k + \frac{m}{M}l)} \quad (4)$$

$$= \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} f_{k,l} \cdot e^{-i2\pi\frac{n}{N}k} \cdot e^{-i2\pi\frac{m}{M}l} \quad (5)$$

$$= \sum_{k=0}^{N-1} \left[\sum_{l=0}^{M-1} f_{k,l} \cdot e^{-i2\pi\frac{m}{M}l} \right] e^{-i2\pi\frac{n}{N}k} \quad (6)$$

Iz zgornje enačbe vidimo, da lahko 2D DFT izračunamo tako, da najprej naredimo 1D DFT vsake vrstice in nato še 1D DFT nad stolpci transformiranih vrednosti. Ta pristop je računsko veliko bolj učinkovit, saj lahko za izračun uporabimo hitro Fourierovo transformacijo (FFT).

Z inverzno diskretno 2D Fourierovo transformacijo lahko transformirano sliko pretvorimo nazaj v prostorsko domeno:

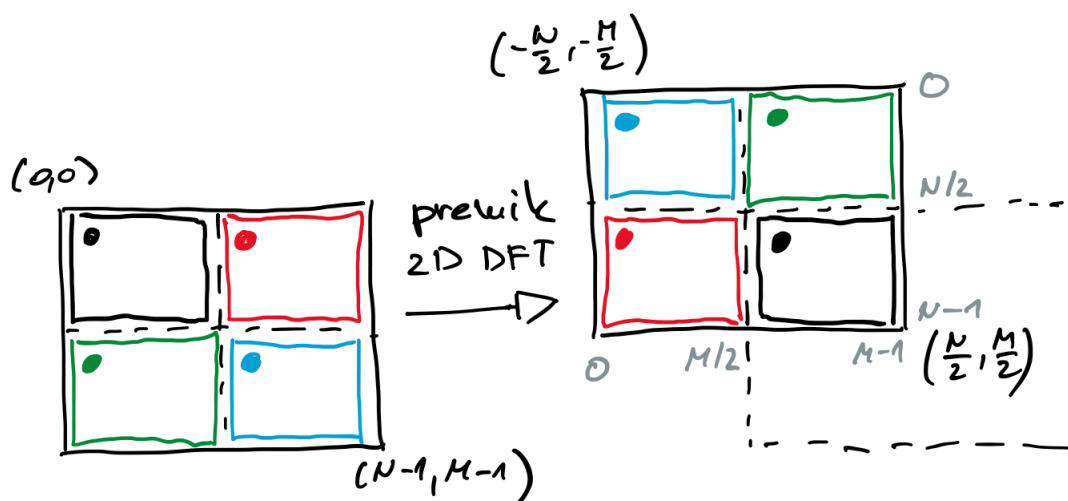
$$f_{k,l} = \frac{1}{NM} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} F_{n,m} \cdot e^{i2\pi(\frac{k}{N}n + \frac{l}{M}m)} \quad (7)$$

Tudi 2D IDFT lahko izvedemo kot vsoto dveh zaporednih 1D IDFT. Postopamo podobno, le da 1D IDFT najprej naredimo nad vsemi stolpci in nato še nad vrsticami transformiranih vrednosti.

V enačbi 1 opazimo, da ima pri transformaciji 1D signala iz časovne v frekvenčno domeno diskretna Fourierova transformiranka periodo N . Periodičnost izkorišča algoritem FFT, ki je zasnovan tako, da Fourierovo transformiranko namesto na intervalu $[-\frac{N}{2}, \frac{N}{2})$ računa na intervalu $[0, N)$. Zato moramo pri analizi koeficiente F_n , $n \in [N/2, N)$ prestaviti na negativni del osi.

Ker za 2D DFT zaporedoma dvakrat izvedemo 1D FFT, moramo enako narediti tudi v tem primeru, le da tokrat v obeh dimenzijah. Pri tem upoštevamo, da je Fourierova transformiranka periodična s periodo N po stolpcih in s periodo M po vrsticah. Razporeditev koeficientov Fourierove transformiranke, ki je primernejša za analizo slike, prikazuje slika 1. Točke na enaki oddaljenosti od sredine, levo-desno in gor-dol, zdaj predstavljajo komplementarne vrednosti v Fourierovi domeni. S tem je tudi obdelava slike bolj intuitivna, saj so na sredini prikazane počasne, na robovih pa hitre spremembe v sliki.

V programskih jezikih polja in tabele običajno definiramo za nenegativne vrednosti, zato raje točko $(0, 0)$ prestavimo proti sredini slike (sive oznake na desni shemi). V programskem jeziku python s knjižnico numpy tak premik enostavno naredimo s funkcijo `roll`.



Slika 1: Za analizo primernejša razporeditev koeficientov Fourierove transformiranke.

Odstranjevanje anomalij v sliki

Po uporabi IDFT se lahko v izhodni sliki pojavijo kompleksne vrednosti. To se zgodi, ker Fourierova transformacija uporablja kompleksna števila. S spreminjanjem vrednosti koeficientov v Fourierovi domeni (na primer z uporabo filtrov) lahko povzročimo, da se imaginarne komponente ohranijo tudi pri transformaciji v prostorsko domeno. Drug vzrok so računske napake, ki se pojavijo zaradi omejene natančnosti računanja. Poleg tega lahko uporaba filtrov povzroči premik vrednosti slikovnih točk izven dovoljenega območja $[0, 1]$. Kompleksnih intenzitet se znebimo tako, da ohranimo samo realno komponento, imaginarno pa zavržemo. Intenzitete dodatno omejimo na dovoljeni razpon s funkcijo

$$\text{clip}(f_{k,l}, a, b) = \begin{cases} a & \text{če } f_{k,l} < a \\ f_{k,l} & \text{če } a \leq f_{k,l} \leq b \\ b & \text{če } f_{k,l} > b \end{cases} \quad (8)$$

V našem primeru mora biti intenziteta slikovnih točk na intervalu $[0, 1]$, zato nastavimo $a = 0$ in $b = 1$.

Visokoprepustni filter

Visokoprepustni filter (*angl.* high-pass filter) je eden bolj preprostih filtrov za obdelavo slik. S filtrom iz predstavitve slike v Fourierovi domeni odstranimo počasne spremembe, hitre pa ohranimo,

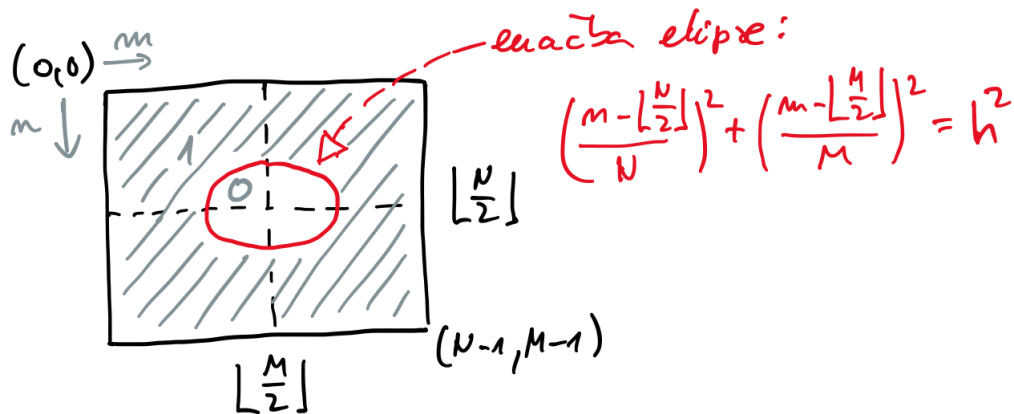
$$H_{n,m} = \begin{cases} 1, & \text{če } \left(\frac{n}{N}\right)^2 + \left(\frac{m}{M}\right)^2 \geq h^2 \\ 0, & \text{drugače} \end{cases} \quad (9)$$

V enačbi nam $\frac{n}{N}$ označuje stopnjo spremembe slike v stolpcu, $\frac{m}{M}$ stopnjo spremembe v vrstici, h pa prag. Če je za dano točko stopnja sprememb v stolpcu in v vrstici večja ali enaka pragu,

vrednost filtra postavimo na 1, drugače pa na 0.

Prag h je ključni parameter pri uporabi filtra v Fourierovi domeni, saj z njim določimo stopnjo sprememb v sliki, ki jih želimo ohraniti oziroma odstraniti. Izbira praga je odvisna od vsebine slike in ciljev obdelave. Če izberemo prenizko vrednost, bomo iz slike odstranili pomembne podrobnosti; če izberemo previsoko vrednost, ne bomo uspeli odstraniti neželenih komponent (na primer šuma ali zamegljenosti). Tako bo za slike z velikimi enakomernimi površinami verjetno bolje izbrati nižji prag, za slike z veliko robovi in teksturo pa višjega. Robovi v sliki so območja hitrih sprememb v intenziteti slike, zato za zaznavanje robov lahko uporabimo visokoprepustni filter.

Filter je enake velikosti kot slika – ima N vrstic in M stolpcev, oblikujemo ga tako, da ima, tako kot premaknjena Fourierova predstavitev slike, središče v točki $(\lfloor \frac{N}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$. Premik lahko upoštevamo že pri gradnji filtra, tako da računamo oddaljenosti od središča slike. Vse, kar moramo narediti, je, da v enačbi 9 levo stran pogoja spremenimo v $\left(\frac{n - \lfloor \frac{N}{2} \rfloor}{N}\right)^2 + \left(\frac{m - \lfloor \frac{M}{2} \rfloor}{M}\right)^2$ in vrednosti poračunamo za vse točke $n \in [0, N-1]$, $m \in [0, M-1]$. Slika 2 prikazuje pripravo filtra.



Slika 2: Visokoprepustni filter

Merjenje podobnosti med slikami z medsebojno informacijo

Uporaba filtrov odstrani določeno količino informacije iz vhodne slike. S pomočjo medsebojne informacije lahko ovrednotimo podobnost med vhodno in obdelano sliko oziroma količino informacije, ki se je ohranila v sliki po filtriranju. Večja vrednost pomeni, da sta sliki bolj podobni.

Medsebojno informacijo računamo po enačbi

$$I(S_V; S_O) = H(S_V) + H(S_O) - H(S_V, S_O) \quad , \quad (10)$$

kjer S_V označuje vhodno sliko, S_O pa obdelano sliko, obe v prostorski domeni. Za izračun medsebojne informacije potrebujemo verjetnosti, ki jih ocenimo iz histogramov svin.

Ker imajo sivinske slike običajno $G = 256$ odtenkov sive, intenzitete vsake slike razvrstimo na G enakomerno razmaknjenih intervalov. Vzemimo, da ima I_v točk v vhodni sliki S_V intenziteto na intervalu $\left[\frac{v}{G}, \frac{v+1}{G}\right)$, $v \in [0, G-1]$. Verjetnost za intenziteto na tem intervalu lahko ocenimo kot

$$p_v = \frac{I_v}{\sum_{r=0}^{G-1} I_r} \quad . \quad (11)$$

Na enak način postopamo tudi za določitev verjetnosti sivin na obdelani sliki.

Vezane verjetnosti (za zadnji člen v enačbi za medsebojno informacijo) dobimo tako, da preštejemo število slikovnih točk I_{vo} , ki imajo intenziteto na vhodni sliki S_V na intervalu $\left[\frac{v}{G}, \frac{v+1}{G}\right)$ in hkrati intenziteto na obdelani sliki S_O na intervalu $\left[\frac{o}{G}, \frac{o+1}{G}\right)$, $v, o \in [0, G-1]$. Vezane verjetnosti cenimo kot

$$p_{vo} = \frac{I_{vo}}{\sum_{r=0}^{G-1} \sum_{s=0}^{G-1} I_{rs}} \quad . \quad (12)$$

Postopek odkrivanja robov v sliki

Celoten postopek odkrivanja robov je sestavljen iz več stopenj. Postopek predpostavlja, da že imamo vhodno sliko pretvorjeno v sivinsko obliko z intenzitetami na območju $[0, 1]$.

1. Izvedemo 2D diskretno Fourierovo transformacijo vhodne slike:
Vhodna slika je v začetku v prostorski domeni, kjer vsaka vrednost predstavlja intenziteto posamezne slikovne točke. Z diskretno Fourierovo transformacijo sliko prevedemo v Fourierovo domeno.
2. Premik Fourierove predstavitve slike v središče:
Po transformaciji so koeficienti, ki predstavljajo počasne spremembe, recimo $(0, 0)$, na robovih slike. Za lažjo analizo Fourierovo predstavitev slike premaknemo tako, da točko $(0, 0)$ prestavimo proti sredini slike $(\lfloor \frac{N}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$.
3. Pripravljanje visokoprepustnega filtra:
Za odkrivanje robov je potrebno odstraniti počasne spremembe v vhodni sliki in ohraniti hitre spremembe. Ustvarimo filter, kjer odstranimo vse stopnje sprememb nižje od praga h .
4. Filtriranje v Fourierovi domeni:
Fourierovo predstavitev vhodne slike pomnožimo s filtrom, kar ohrani le hitre spremembe v intenziteti (robove). Dobimo Fourierovo predstavitev obdelane slike,

$$F'_{n,m} = F_{n,m} \cdot H_{n,m} \quad , \quad n \in [0, N-1], m \in [0, M-1] \quad . \quad (13)$$

5. Obraten premik Fourierove predstavitve:
Po obdelavi Fourierovo predstavitev obdelane slike premaknemo v obratni smeri kot Fourierovo predstavitev vhodne slike tako, da se točka $(\lfloor N/2 \rfloor, \lfloor M/2 \rfloor)$ premakne v točko $(0, 0)$ na robu slike.

6. Inverzna Fourierova transformacija (IDFT):

Z inverzno diskretno Fourierovo transformacijo Fourierovo predstavitev obdelane slike prevedemo v prostorsko domeno. Dobimo obdelano sliko s poudarjenimi robovi.

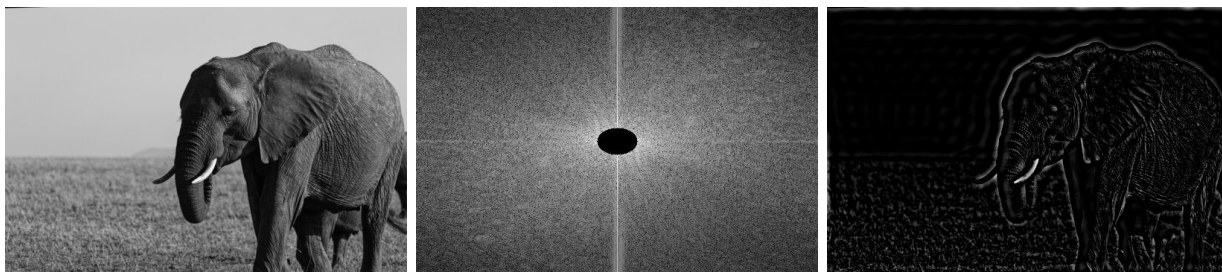
7. Odstranjevanje anomalij:

Da obdelano sliko lahko narišemo, odstranimo imaginarne komponente in sliko omejimo na območje $[0, 1]$.

8. Primerjava vhodne in obdelane slike:

Izračunajmo medsebojno informacijo med vhodno in obdelano sliko.

Slika 3 prikazuje vhodno sliko, realni del Fourierove predstavitve obdelane slike in obdelano sliko za vrednost praga $h = 0.05$. Na srednji sliki se lepo vidi, da je filter vse točke v bližini točke $(0, 0)$ (še v centru slike) postavil na 0 (črna barva).



Slika 3: Vhodna slika, realni del Fourierove predstavitve obdelane slike in obdelana slika.

Naloga

V datoteki `naloga4.py` v jeziku python napišite funkcijo z imenom `naloga4`, ki izračuna:

- medsebojno informacijo med vhodno in obdelano sliko.

Vhodna argumenta funkcije sta

- slikovne točke vhodne slike v obliki matrike numpy (`slika`) in
- prag visokoprepustnega filtra (`prag`).

Prototip funkcije

```
import numpy as np
def naloga4(slika: np.array, prag: int) -> float:
    """
    Poenostavi sliko

    Parameteri
    -----
    slika : numpy array
        vhodna slika
    prag : int
        najmanj "sa frekvenca, ki se ohrani na sliki

    Vrnjena vrednost
    -----
    MI : float
        medsebojna informacija
    """

    MI = float('nan')
    return MI
```

Testni primeri

Na učilnici se nahajajo tri testne slike, za katere imate podane tudi medsebojne informacije. Podatki so podani v obliki datotek `.json`. Priloženo imate tudi funkcijo `test_naloga4`, ki jo lahko uporabite za preverjanje pravilnosti rezultatov. Pri testiranju vaših funkcij upoštevajte naslednje omejitve:

- rezultat je pravilen, če se od danega razlikuje za manj kot 10^{-4} ,
- izvajanje funkcije je časovno omejeno na 30 sekund.
- Vaš program lahko uporablja samo tiste pakete, ki so del standardne knjižnice python 3.12 (<https://docs.python.org/3.12/library/>), in paket numpy. Na sistemu za preverjanje vaših rešitev drugi paketi niso nameščeni.

Namigi

Pri reševanju naloge vam bodo koristile naslednje funkcije iz paketa numpy:

- `fft.fft` in `fft.ifft`,
- `roll`,
- `clip`,
- `ogrid`,
- `histogram` in `histogram2d`,
- `sum` in `log2`.